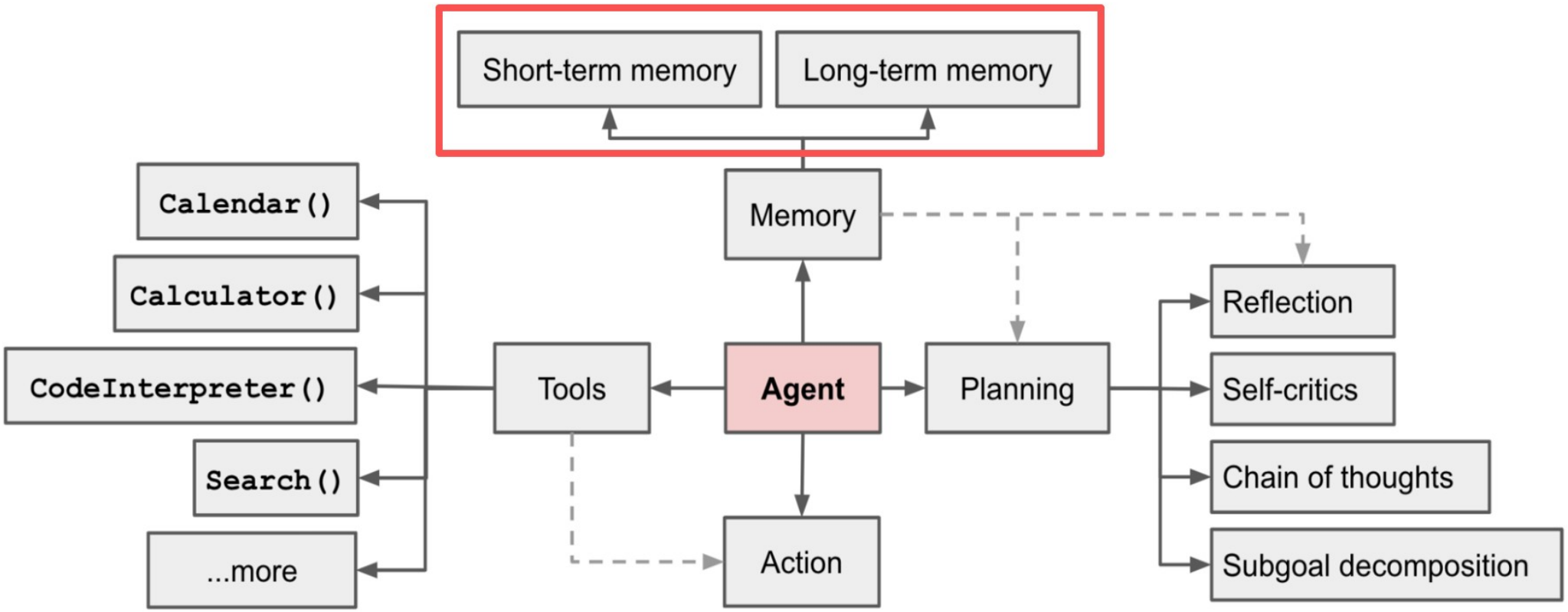
通用人工智能创新实践课 -Agent 的知识与记忆

记忆与智能体



RAG　　Memory　　Knowledge

Agent Memory

RAG

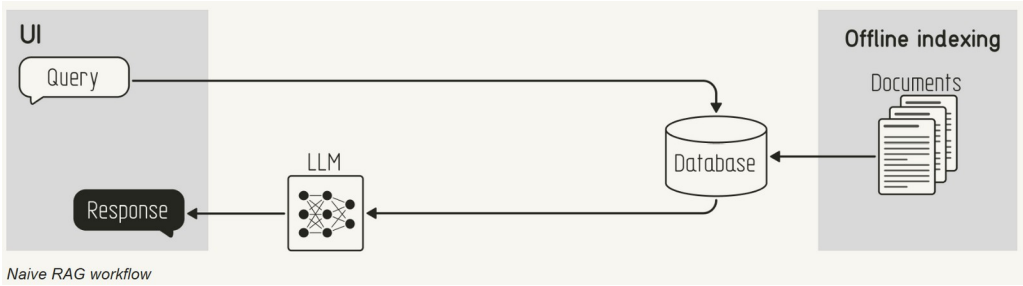
- 信息离线存储，支持手动更新删除
- 一次性检索

Agentic RAG

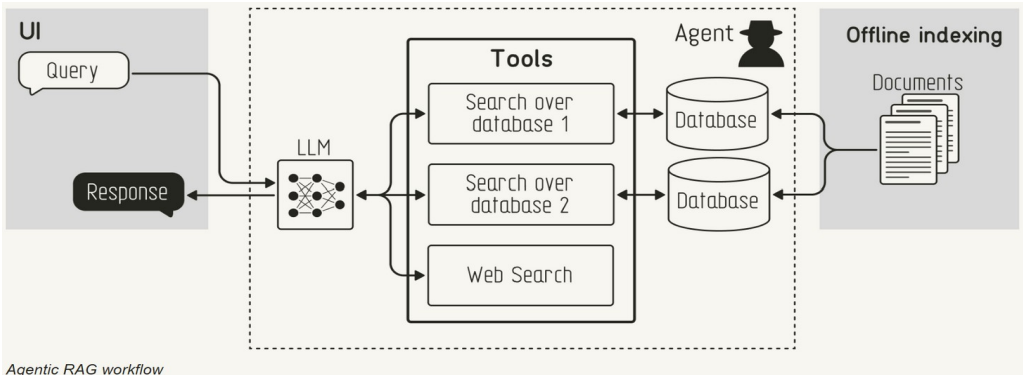
- 信息离线存储，支持手动更新删除
- 由工具或推理引擎发起动态检索

Memory in Agent

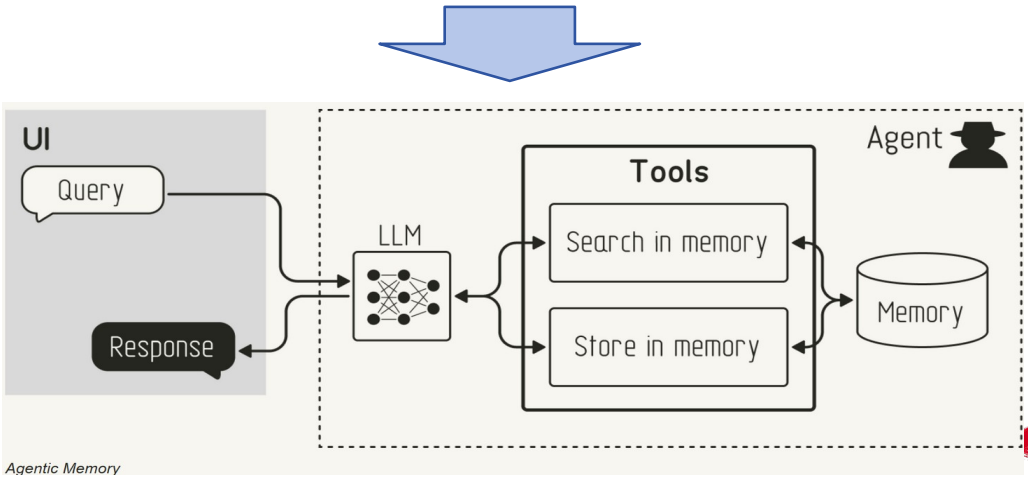
- 信息动态存储，动态更新删除
- 由工具或推理引擎发起动态检索



Naive RAG workflow



Agentic RAG workflow



Agentic Memory

Native RAG

离线索引

- chunk
- document encode
- store

在线检索

- query encode
- search

在线生成

- prompt
- generation

```
## 离线索引阶段

# 1. 把文档切成小块，chunk_size=500 表示每块 500 字，
#    chunk_overlap=50 表示相邻块之间重叠 50 字，避免关键信息被切断
chunks = split_into_chunks(documents, chunk_size=500, chunk_overlap=50)

# 2. 把每个小块转成向量，连同原文一起存入向量数据库
for each chunk in chunks:
    vector = embedding_model.encode(chunk)    # 调用 Embedding 模型编码
    vector_store.insert(vector, chunk)        # 向量和原文一起入库


## 在线查询生成阶段

# 1. 用同一个 Embedding 模型把用户问题也转成向量
query_vector = embedding_model.encode(user_query)

# 2. 在向量库里搜最相似的 Top 5 文档块
top_k_chunks = vector_store.search(query_vector, k=5)

# 3. 把检索到的文档块拼进 Prompt，作为参考资料
prompt = "基于以下参考资料回答问题：\n" + join(top_k_chunks) + "\n问题：" + user_query

# 4. 交给大模型生成最终回答
answer = LLM.generate(prompt)
```

RAG PseudoCode

RAG - Chunk

必要性

- 嵌入式模型，有严格的输入长度上限
- 生成模型，有上下文长度限制， chunk size 过大，会影响信息广度

分片大小

- 过小：信息碎片化，上下文缺失
- 过大：信息损失，噪声过大（ Lost in the middle ）、主题稀释
- 经验值 200~1024

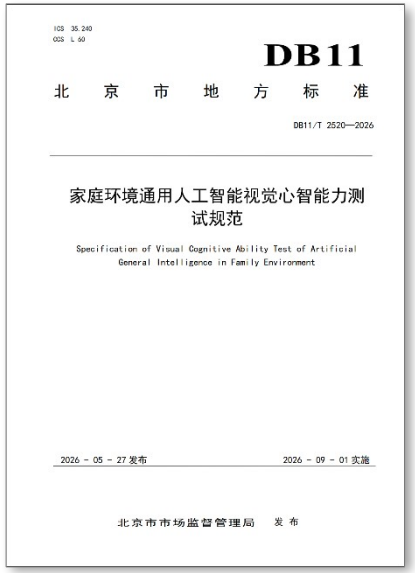
分片策略

- 固定大小，段落 / 分隔符分割 + 智能合并
- 语义分块，在语义主题发生显著变化的地方进行切分。计算相邻句子的语义距离。
- 基于文档结构
  - MarkDown 、 JSon
  - 元数据注入
- 滑动窗口， Overlapping Chunking 15%~20%
- 父子文档分块



	A (序	B (值班	C (装置名	D (上报时	E (汇报人	F (存在问题)	G (责任	H (监督部	J (处置情况)
21	14	[illegible]	[illegible]	1.11 14:11	[illegible]	50万装置吸收塔差压PDI-1102中控显示乱跳	生产办	HSE办	已处理
23	496	[illegible]	[illegible]	12.07 19:58	[illegible]	300万液硫泵凝结水保温管线有个法兰漏	生产办	HSE办	已处理
24	263	[illegible]	[illegible]	6.18 21:26	[illegible]	300万空压机A台机组压力0.44，怀疑在漏气	生产办	HSE办	（18日陶主任通知暂时切换另一台，配电关注温度；停运一台干噪器）21日庞增诚：厂家已维护300万空压机A台。通知操作班启运A台。
25	250	[illegible]	[illegible]	6.7 4:50	[illegible]	300万风机B台故障，引起回收联锁。启运A台装置于5:00恢复正常后，再次启运B台风机，正常运行几分钟后手动停机。	生产办	HSE办	,10日16:26日杨斌：已处理，正常使用
26	67	[illegible]	[illegible]	1.23 15:36	[illegible]	300万脱水废气伴热管线上一个疏水阀阀体上的螺栓漏蒸汽，紧了一下还是漏，阀可以正常工作。	生产办	HSE办	1.26日曾思逊：待料维修
27	502	[illegible]	[illegible]	12.10 8:20	[illegible]	300万凝结水罐上水调节阀旁路阀出口端法兰连接处漏	生产办	HSE办	10日14:30杨松烨：紧了下不得行，现在暂时还处理不了，两边旁不了。 10日15:20罗主任：目前在滴，监护使用。
28	459	[illegible]	[illegible]	11.10 11:07	[illegible]	50万脱硫贫液后冷封头漏溶液	生产办	HSE办	10日15:45杨松烨：紧固了下，观察使用
29	150	[illegible]	[illegible]	3.10 11:40	[illegible]	成型单元凉水塔平台围栏基脚锈蚀断裂4根	生产办	HSE办	10日庞增诚：需要花时间，外面去预制了现场来安装
30	389	[illegible]	[illegible]	9.20 10:48	[illegible]	300万主胺泵A后端散热片漏机油	生产办	HSE办	11:50夏泽雨：看了是风扇有点浸油，目前不管它，严重了以后换油封。（张林军已知晓）
31	139	[illegible]	[illegible]	2.23 11:40	[illegible]	300万制氮机A台屏幕显示不亮	生产办	HSE办	11：53庞增诚：需待料整改
32	252	[illegible]	[illegible]	6.10 16:44	[illegible]	污水混合池进水阀门阀芯掉了	生产办	HSE办	11日12:05杨斌：已处理
33	465	[illegible]	[illegible]	11.11 14:05	[illegible]	50万脱硫贫液后冷封头漏溶液，昨天弄了还在漏	生产办	HSE办	11日15:52杨松烨：垫子遭了，又紧了下，还有点浸；要换需停产。
34	464	[illegible]	[illegible]	11.11 14:05	[illegible]	80万湿净化气分离器排污阀角阀漏	生产办	HSE办	11日15:52杨松烨：阀圈的问题，要换需停产。
35	463	[illegible]	[illegible]	11.11 9::23	[illegible]	80万蒸汽流量FIC-1105卡起了	生产办	HSE办	11日9:55徐晓芳：输出值超过满量程了，莫法弄。 11 日14:59丁大林：80万主蒸汽流量超过计量孔板设计量程上限，无法显示，孔板计量正常
36	485	[illegible]	[illegible]	11.29 22:02	[illegible]	80万酸空冷管束漏液（12月18日18:09龙德林：漏得大了，有两处漏点）	生产办	HSE办	12月18日罗主任：按照之前的监护方案执行，加强监控，做好安全防护
37	368	[illegible]	[illegible]	9.11 15:10	[illegible]	污水泵3/B不上量	生产办	HSE办	13日11：21张林军：泵3现在用起的，连泵头有问题，每次用都要灌引水，要换一个连泵头。（已联系姚霖）
38	33	[illegible]	[illegible]	1.13 9:48	[illegible]	污水外排泵A台漏，停起的。	生产办	HSE办	13日顺辉易强：待料，材料到了马上处理。(22日易强：材料还未到，处理好了回复)
39	249	[illegible]	[illegible]	6.4 10:18	[illegible]	锅炉循环水泵A台电机后端异响	生产办	HSE办	14:06杨铎：已处理，观察运行。

文档 1



ICS 35.240
CCS L 60

DB11

北 京 市 地 方 标 准

DB11/T 2520—2026

家庭环境通用人工智能视觉心智能力测试规范

Specification of Visual Cognitive Ability Test of Artificial General Intelligence in Family Environment

2026 - 05 - 27 发布　　2026 - 09 - 01 实施

北京市市场监督管理局　发布

文档 2

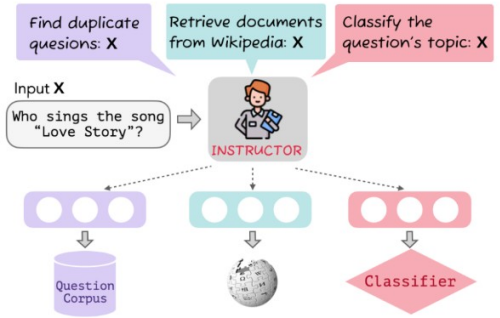
RAG - Embedding

模型选择

- RAG 关键维度
  - Retrieval
  - Reranking
  - Instruction Reranking
- 经验选择
  - 中文： Qwen3-Embedding 、 BGE-M3
  - 多语言 / 多模态： Cohere embed
  - Coding： Voyage

指令重排

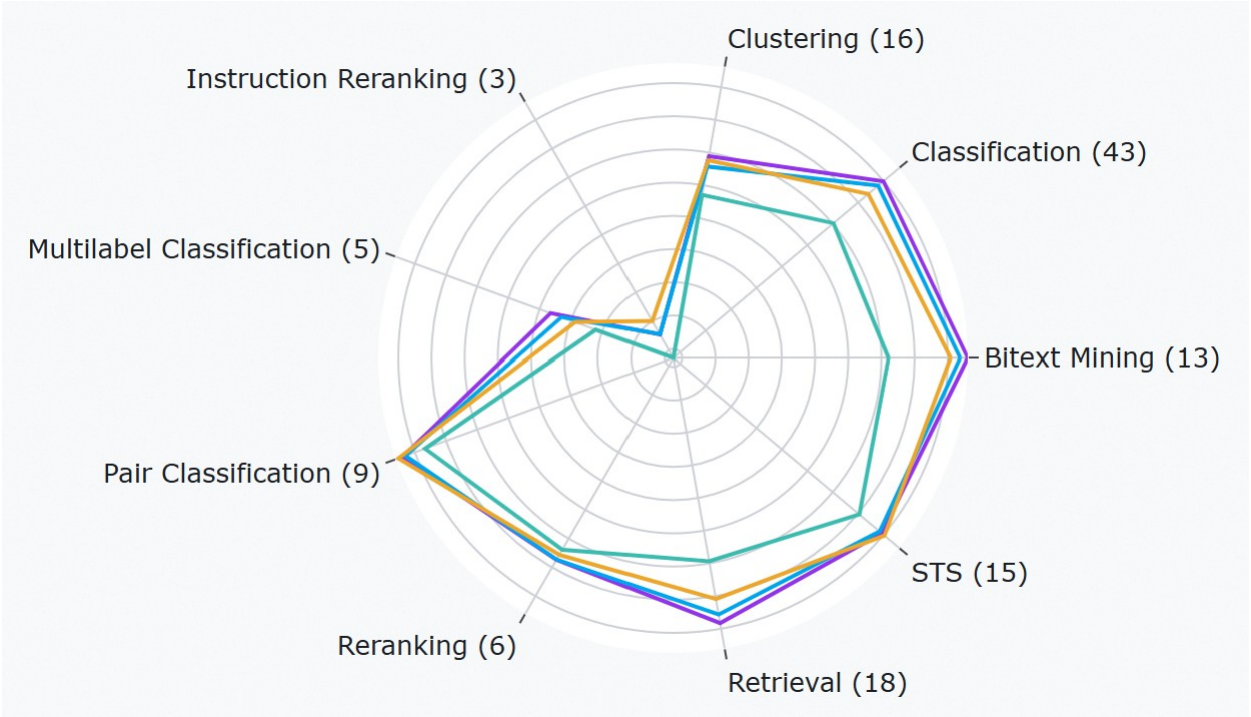
- 通过额外指令引入业务规则，丰富上下文信息



权威榜单

- MTEB / C-MTEB

```python
resp = dashscope.TextEmbedding.call(
    model="text-embedding-v4",
    input="机器学习的相关论文",
    text_type="query",  # 声明为查询
    instruct="Given a research paper query, retrieve relevant research paper"
)
```



model compare

RAG - Storage

向量数据库	核心特点	典型使用场景	部署形态
Milvus	云原生、分布式架构，可支撑十亿级向量规模；支持多种索引与 GPU 加速，内置可视化运维工具 Attu 。	RAG 知识库问答、以图搜图与多模态检索、推荐系统、分子结构检索	开源自托管 · Zilliz Cloud
Pinecone	全托管 Serverless ，免运维、毫秒级延迟；内置元数据过滤、命名空间与用量监控，按量计费易扩展。	生产级语义搜索与 RAG 、个性化推荐、异常检测、智能客服问答	商业 SaaS 云服务
Qdrant	Rust 编写、性能与并发能力强；支持 Payload 过滤、稀疏与稠密向量混合检索、多租户隔离。	需精确过滤的电商 / 内容检索、实时推荐、风控与欺诈检测	开源自托管 · Qdrant Cloud
Weaviate	开源向量库，内置向量化、重排序等 AI 模块； GraphQL / REST API ，原生混合检索（ BM25 + 向量）。	企业知识库问答、文档与图像语义检索、 Agent 工具记忆	开源自托管 · Weaviate Cloud
Chroma	轻量嵌入式向量库， Python / JS 原生 API ；与 LangChain 、LlamaIndex 深度集成，本地几分钟即可运行。	RAG 原型与教学实验、个人 / 小团队本地知识库、快速 Demo	开源 · 本地部署为主
pgvector	PostgreSQL 扩展，新增 vector 类型与 HNSW / IVFFlat 索引；可复用 SQL 、事务、权限与备份生态。	已有 PG 业务的 AI 化升级、中小规模语义检索、快速 MVP 验证	开源 · 随 PostgreSQL 部署
Elasticsearch	搜索引擎原生支持 dense_vector 与 kNN 检索，可与全文检索组合实现 Hybrid Search 混合查询。	日志 / 文档检索与语义搜索融合、站内搜索与内容理解	开源自托管 · Elastic Cloud

Faiss 、 Annoy 提供向量检索算法库； MongoDB Atlas 、 Redis 、 SQLite （ sqlite-vec ）等则在传统存储之上内置了向量检索能力。

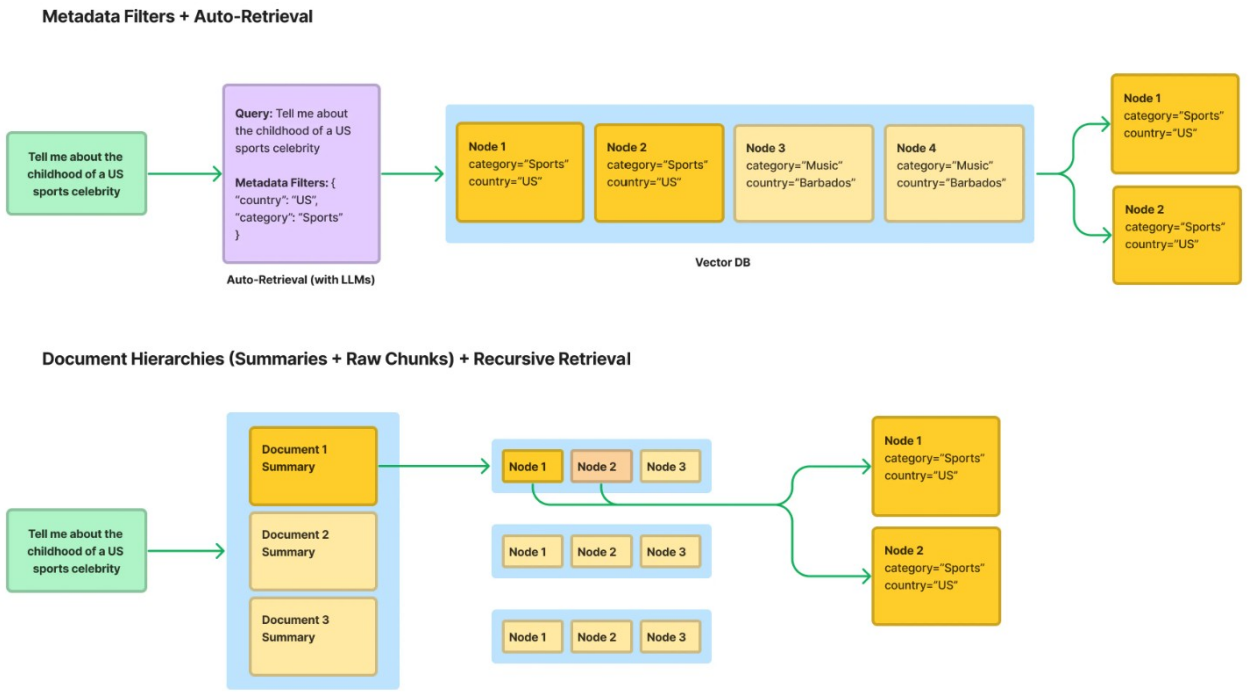
RAG - Index

参数级优化

- 索引类型选择
  - FLAT 暴力搜索， 100% 召回。适合数据规模小，精度要求高
  - IVF(倒排文件索引) ，通过聚类建立桶，先找到最匹配的桶，然后在桶内进行精确搜索。召回率略低，适合高吞吐
  - HNSW(分层可导航小世界图) ，分层图结构，高层稀疏定位大致区域，底层稠密用于精确检索。内存大，构建慢，适合低延迟
  - DiskANN 为在磁盘上运行而优化的图索引，适合海量数据
- 批量读写、优化召回字段降低系统、网络开销
- 向量归一化、强制段合并、 GPU 模式优化计算性能

架构级优化

- 结构化索引，为文本块附加结构化的元数据。先元数据过滤再搜索
- 创建两个独立索引，摘要索引和内容索引

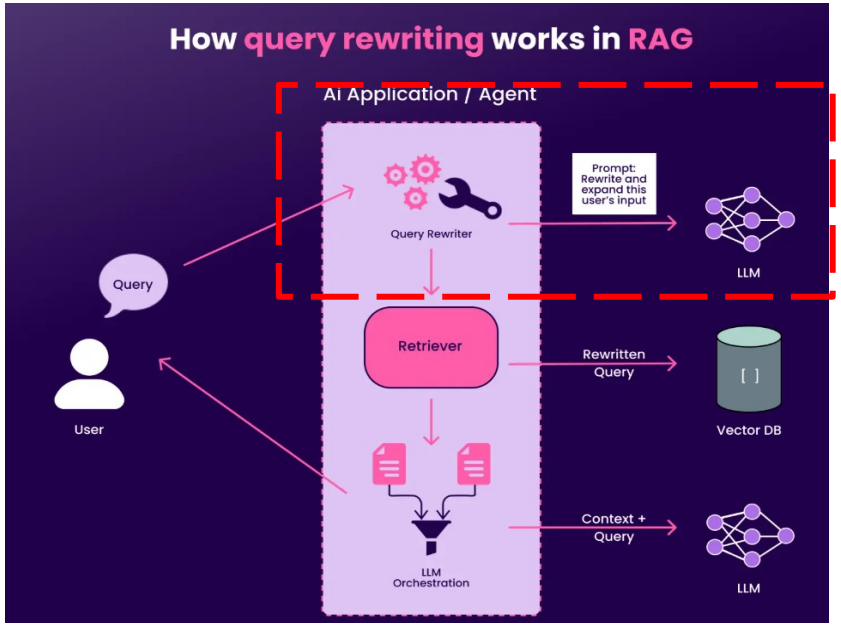


结构化索引

RAG - Query Rewriting/Construction

Rewriting

- 查询扩展，把相关术语或概念添加到原始查询中
- 查询分解，复杂查询拆解为子查询
- 查询意译，对同一查询使用不同的措辞
- 多查询生成，多个重写版本的单一用户请求。问题的不同视角。
- 退步提示，首先生成一个更广泛或更通用的问题。
- 假设性文档嵌入（HyDE），先生成一个“假设性”的、能够回答该查询的文档。然后根据假设性文档找到与之最相近最相似的真实文档



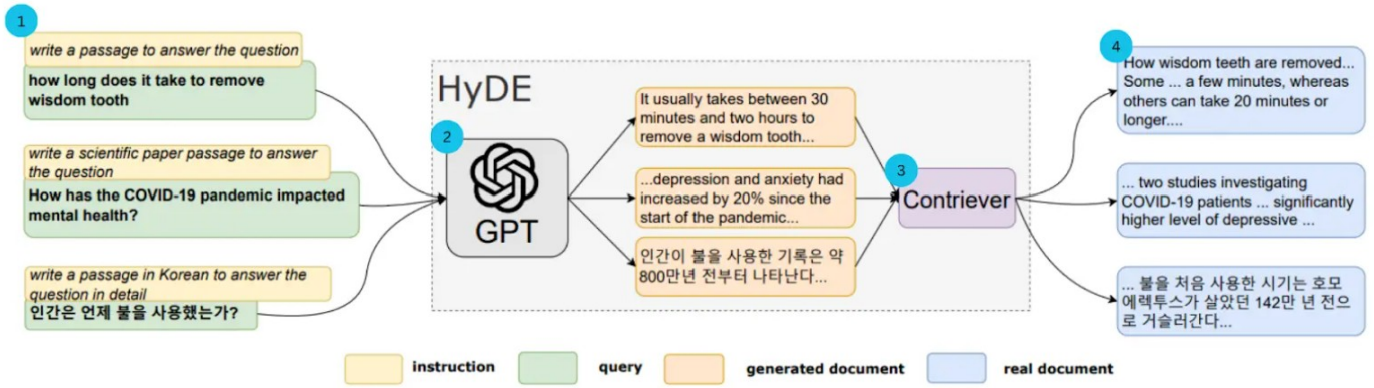
Rewriting in RAG

Route

- 动态选择最合适的处理路径

Construction 检索结构化数据

- Text2SQL
- Text2Cypher



HyDE

RAG - Search

召回阶段

- 稀疏索引（TF-IDF/BM25），基于词频统计。可解释但存在“词汇鸿沟”

$$Score(D,Q) = \sum_{t \in Q} IDF(t) \times \frac{f(t,D) \cdot (k_1 + 1)}{f(t,D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{avgD}\right)}$$

- 密集索引， Bi-Encoder （BGE/OpenAI text-embedding 等），将数据压缩到低维、连续、有语义含义的向量空间，泛化能力强，可解释性差。

融合阶段 混合检

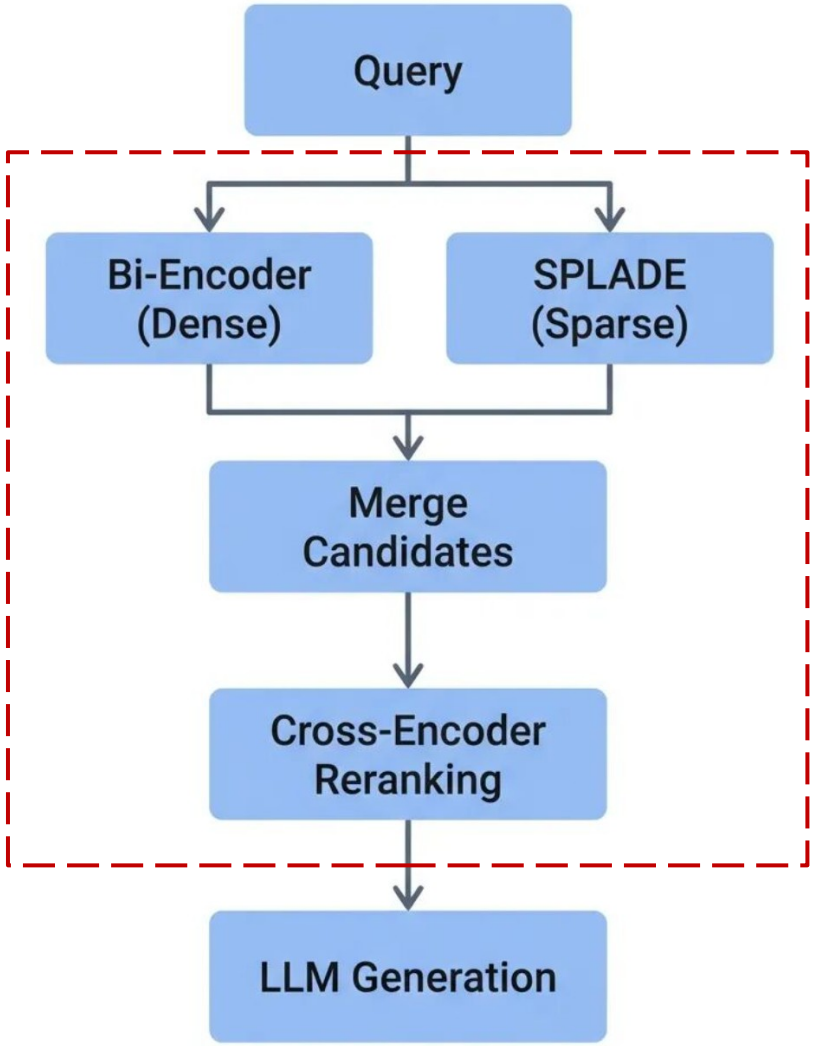
$$Hybrid_{score} = \alpha \cdot Dense_{score} + (1 - \alpha) \cdot Sparse_{score}$$

- 加权线性组合
- 倒数排序融

$$RRF_{score}(d) = \sum_{i=1}^{k} \frac{1}{rank_i(d) + c}$$

重排阶段

- 解决召回阶段”粗而不精”的问题，减少噪声，解决长尾问题
- 常见算法
  - Cross-Encoder （BGE-Reranker/Cohere Reranker），把查询和候选文档拼接成一个输入序列，由 transformer 架构计算内部相关性，无法预计算
  - ColBERT ，查询和文档都拆成 token 级向量，交叉匹配计算相似度，



search in RAG

Gragh RAG

Gragh RAG

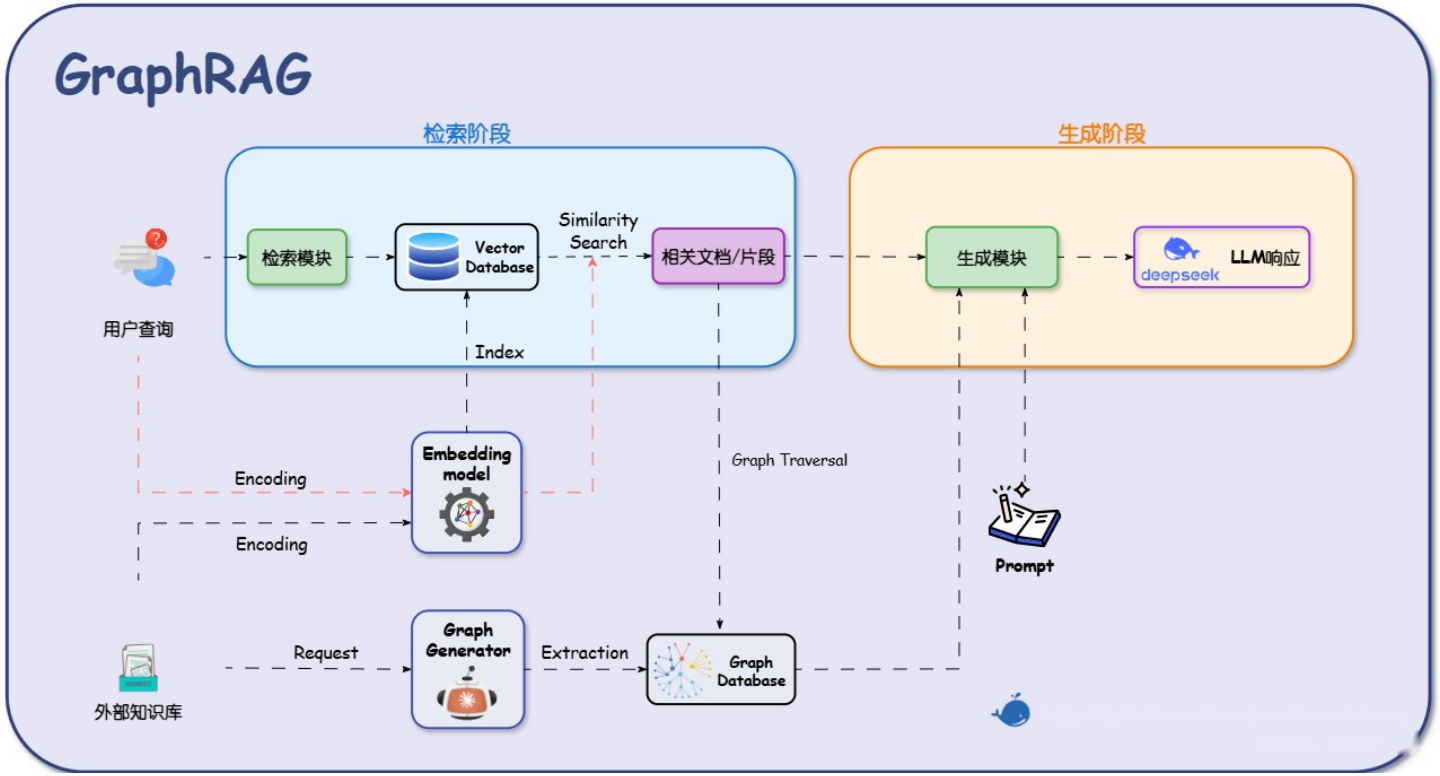
通过利用知识图谱的显式语义关系和图结构优势，提供更精准的上下文检索和更强的推理能力。

解决问题

- 关系理解缺失
- 上下文碎片化
- 跨文档联结能力弱

基本流程

- 知识图谱构建：知识抽取 -> 图谱融合 -> 索引构建
- 图谱检索：
  - local： 实体定位 -> 子图探索 -> 结构化证据
  - global： 社区摘要 -> 内容聚合 -> 答案生成
  - DRIFT： 实体定位 -> 社区扩展 -> 答案生成
- 增强生成：提示设计 -> 证据融合 -> 约束回答
- **开源框架**
- GraphRAG （microsoft）
- LightRAG
- Flexible RAG



Gragh RAG

Corrective-RAG

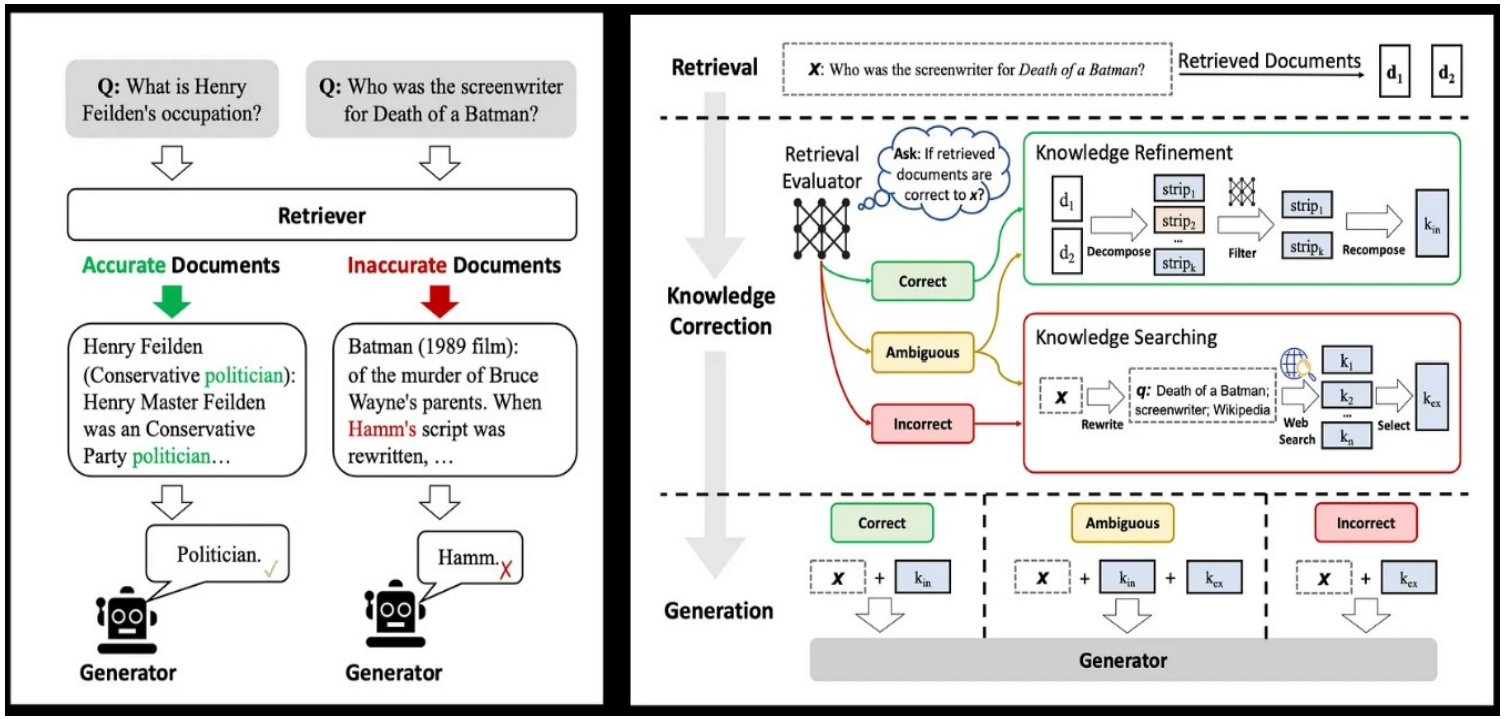
传统 RAG 无法验证检索的正确性， Corrective-RAG 在 Generation 之前增加 Evaluation 步骤

解决问题

- 避免检索到不相关、过时、错误的文档，进而生成幻觉、错误回答。

关键步骤

- 检索（Retrieve)
- 评估（Assess)，判断每个文档与查询的相关性，依赖于外部评估器
- 行动（Act)
  - Correct ->Knowledge Refinement
  - Incorret -> Query Rewriting
  - Ambiguous -> Knowledge Search



C-RAG

Agentic RAG

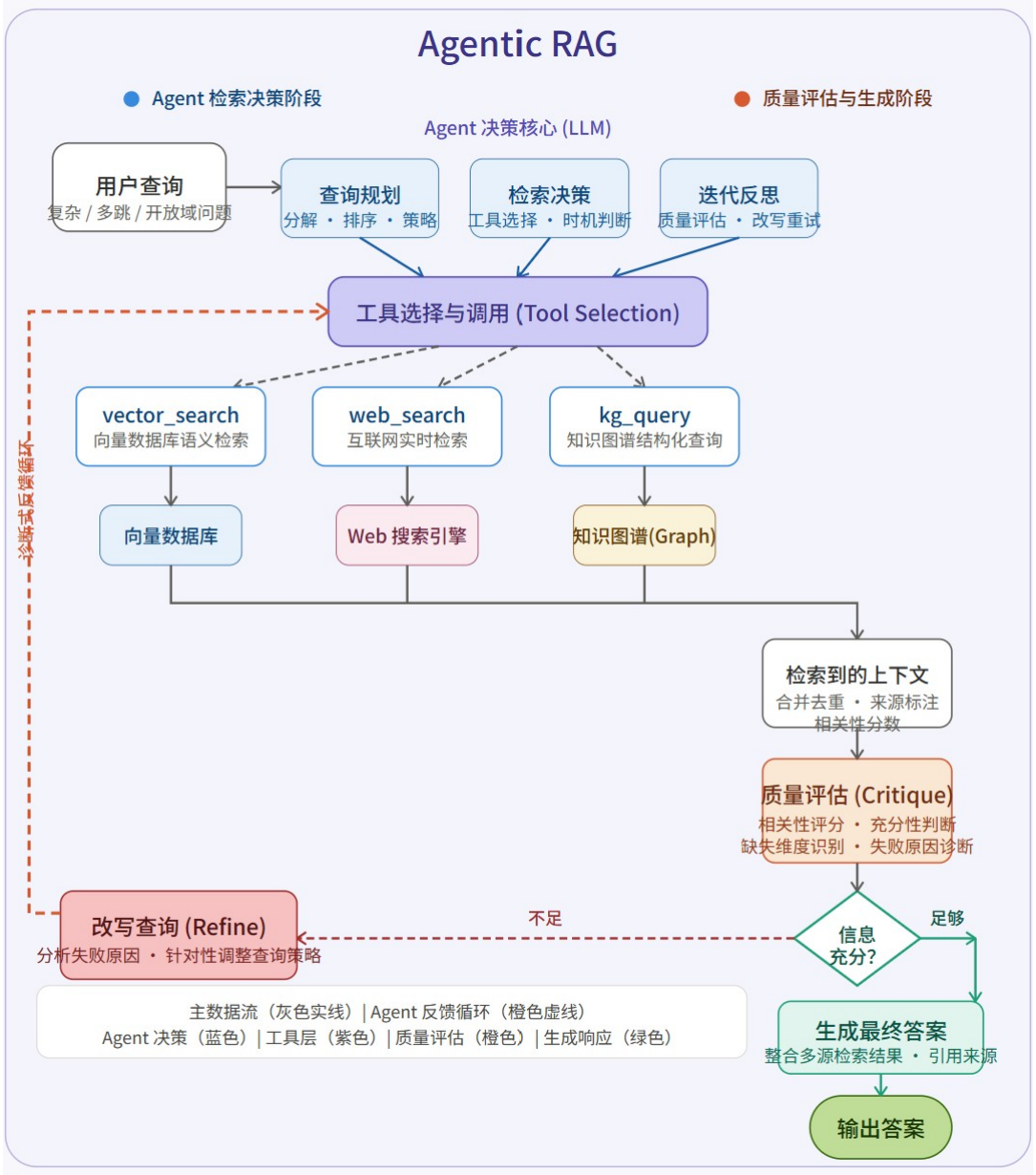
传统 RAG 一个固定调用流程， Agentic RAG 是一个需要决策的认知过程

核心能力

- 检索决策，何时检索、用什么工具、如何组织参数
- 查询规划，拆解子任务、决定检索顺序
- 质量自评，评估结果的相关性和充分性
- 迭代改写，诊断式改写查询

```python
def _critique(self, state):
    prompt = f"""评估检索质量。原问题：{state['original_query']}
    检索结果：{state['collected_docs'][-5:]}
    输出JSON：relevance_score(0-10), sufficiency_score(0-10),
    is_sufficient(bool), failure_reason, suggested_rewrite"""
    response = self.llm.invoke(prompt)
    state["critique_result"] = json.loads(response.content)
    return state


def _refine_query(self, state):
    failure = state["critique_result"]["failure_reason"]
    prompt = f"查询'{state['current_query']}'失败原因：{failure}。针对性改写。"
    response = self.llm.invoke(prompt)
    state["current_query"] = response.content.strip()
    return state
```



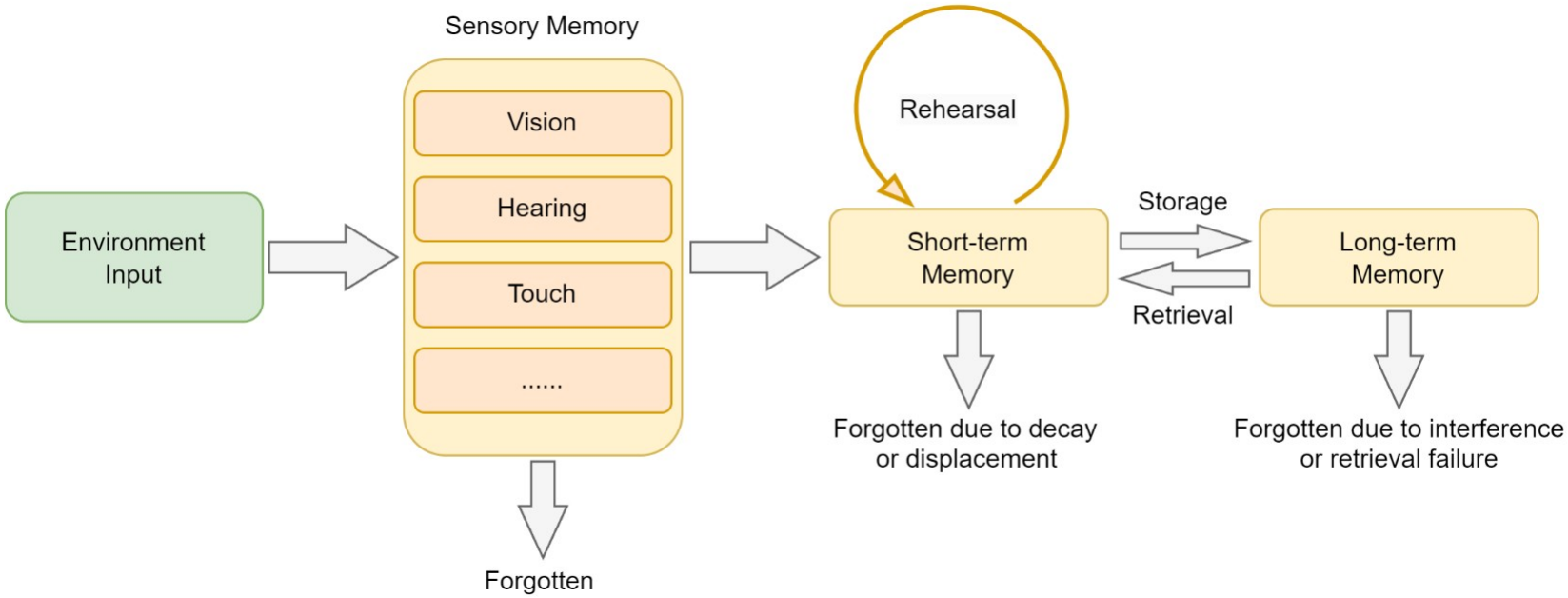
Agentic RAG

人脑记忆层级

感觉记忆（Sensory Memory）
- 持续时间极短（0.5-3 秒）
- 容量巨大
- 负责暂时保存感官接收到的所有信息

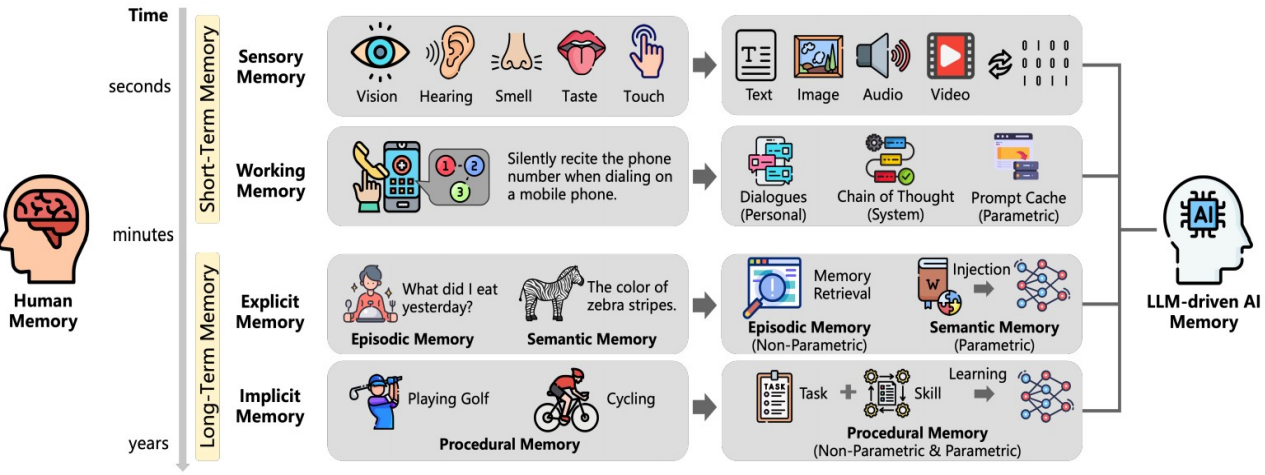
工作记忆（Working Memory）
- 持续时间短（15-30 秒）
- 容量有限（7±2 个项目）
- 负责当前任务的信息处理

长期记忆（Long-Term Memory）
- 持续时间长，容量几乎无限
- 程序性记忆：技能和习惯
- 陈述性记忆：可以用语言表达的知识
  - 语义记忆：一般知识和概念
  - 情景记忆：个人经历和事件



人类记忆系统的层次结构

Agent Memory



Human memory vs Agent memory

	信息存储	信息检索	信息更新
RAG	离线操作	一次性检索	手动更新
Agentic RAG	离线操作	动态按需检索	手动更新
Memory	在线操作	动态按需检索	动态更新

RAG vs Agentic RAG vs Memory

Agent Memory

Storage

- 稀疏存储、密集存储、图存储
- 多模态存储， Sensory Memory

Retrieval

- 检索时机：程序式、委托式、补充式（快慢系统）
- 检索路由：记忆分级、多智能体架构

Evolution

- 融合：逐级融合、按主题融合
- 更新：语义一致性、双轨更新

Forget

- 基于时间： FIFO 、模拟衰减强度
- 基于频率： LRU 、 BloomFilter
- 基于重要性：语义级评估， MemTool

```
while user_active:
    user_message = receive_message()

    # READ
    long_term_context = search_long_term_memory(user_id, user_message)
    short_term_context = read_short_term_memory(session_id)

    # BUILD PROMPT (retrieved memories are formatted into text and
    # injected into the system prompt, not sent as separate messages)
    messages = [system_prompt_with(long_term_context, short_term_context), user_message]

    # LLM LOOP (may call tools multiple times before the final answer)
    while True:
        response = call_llm(messages, tools)
        if response.is_tool_call:
            tool_result = call_tool(response.tool_name, response.tool_args)
            messages.append(response)
            messages.append(tool_result)
        else:
            break

    # WRITE
    write_short_term_memory(session_id, user_message, response)
    if is_worth_keeping(user_message, response):
        write_long_term_memory(user_id, user_message, response)

    send_to_user(response)
```

Memory PseudoCode

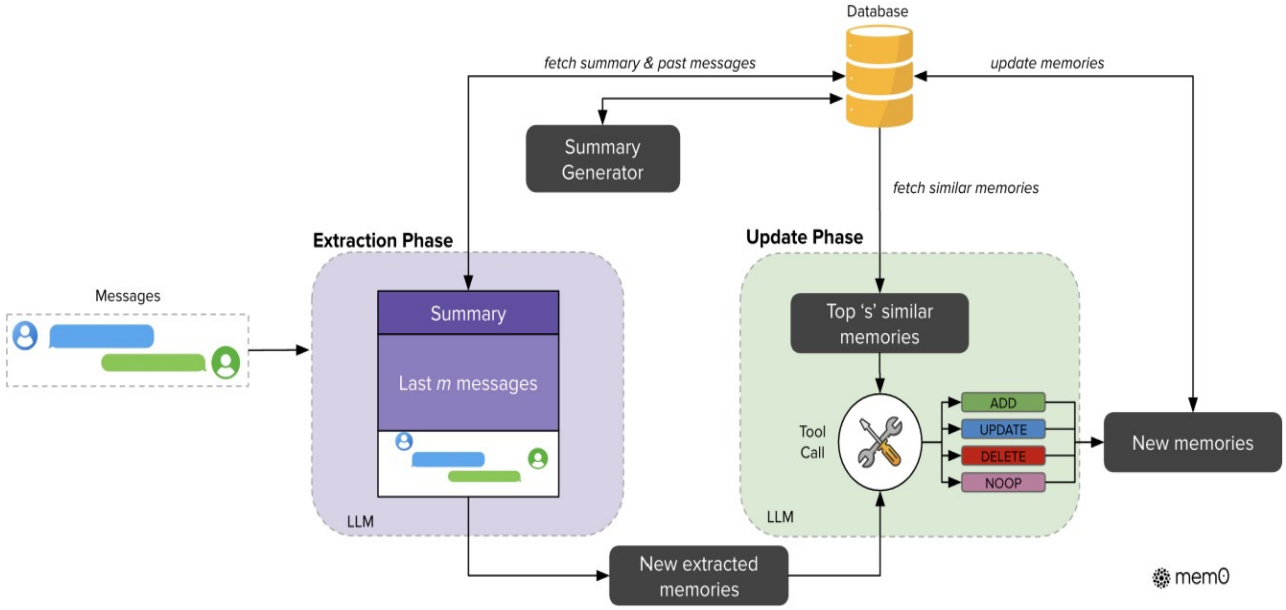
Memory 框架 - Mem0

Mem0

- 基础接口： add/update/history/get_all/search/delete
- 生态： 27+ embedding database； 15+ embedding model； 5+ reranker model
- MCP/Tool： search/update/save

Mem0g

- 图结构，表示顺序、进程或实体间的联系



Mem0 Framework

```python
def retrieve_context(query: str, user_id: str) -> List[Dict]:
    """Retrieve relevant context from Mem0"""
    try:
        memories = mem0.search(query, filters={"user_id": user_id})
        memory_list = memories['results']

        serialized_memories = ' '.join([mem["memory"] for mem in memory_list])
        context = [ ... ]
        return context
    except Exception as e: ...

def generate_response(input: str, context: List[Dict]) -> str:
    """Generate a response using the language model"""
    chain = prompt | llm
    response = chain.invoke({
        "context": context,
        "input": input
    })
    return response.content

def save_interaction(user_id: str, user_input: str, assistant_response: str): ...


def chat_turn(user_input: str, user_id: str) -> str:
    # Retrieve context
    context = retrieve_context(user_input, user_id)
    # Generate response
    response = generate_response(user_input, context)
    # Save interaction
    save_interaction(user_id, user_input, response)

    return response
```

Func Call

```python
@function_tool
def search_memory(ctx: RunContextWrapper[UserContext], query: str) -> str:
    """Search through past learning history and memories."""
    memories = mem0.search(
        query,
        filters={"user_id": ctx.context.user_id},
        limit=5
    )
    if memories and memories.get("results"):
        return "\n".join([
            f"- [ID: {m['id']}] {m['memory']}" for m in memories["results"]
        ])
    return "No relevant memories found."
```

Tool Use

Memory 框架 - memobase

设计理念

- Memory for User： profile
- Time-aware： user-event
- Controllable Memory： flexible configuration

核心概念

- Project/User/Profile/Event/Data/Buffer

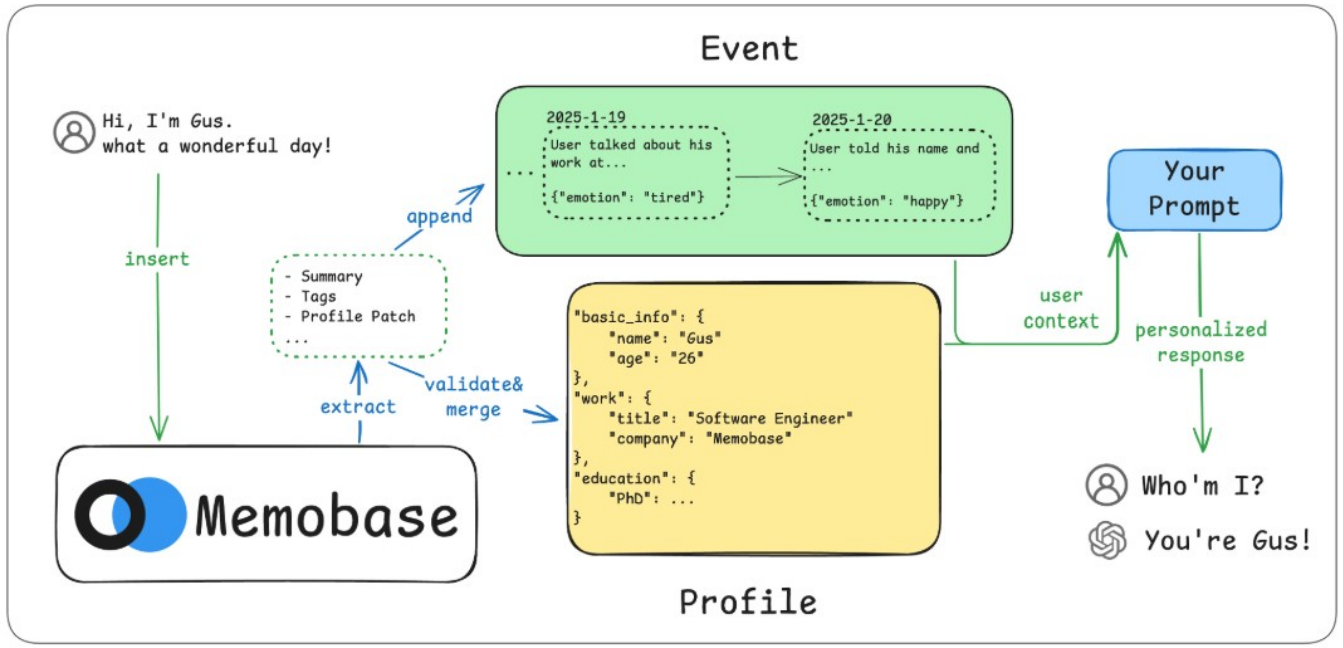
```python
async def llm_node(
    self,
    chat_ctx: llm.ChatContext,
    # ... other params
) -> AsyncIterable[llm.ChatChunk]:
    # Ensure Memobase is reachable
    assert await mb_client.ping(), "Memobase is not reachable"

    # Get or create the user in Memobase
    user = await mb_client.get_or_create_user(string_to_uuid(self.user_name))

    # Insert new messages into memory
    if len(chat_ctx.items) > self.chat_log_index:
        new_messages = chat_ctx.items[self.chat_log_index : len(chat_ctx.items) - 1]
        if len(new_messages):
            blob = ChatBlob(
                messages=[
                    {"role": m.role, "content": m.content[0]}
                    for m in new_messages
                    if m.role in ["user", "assistant"]
                ]
            )
            await user.insert(blob)
            await user.flush()
            self.chat_log_index = len(chat_ctx.items) - 1

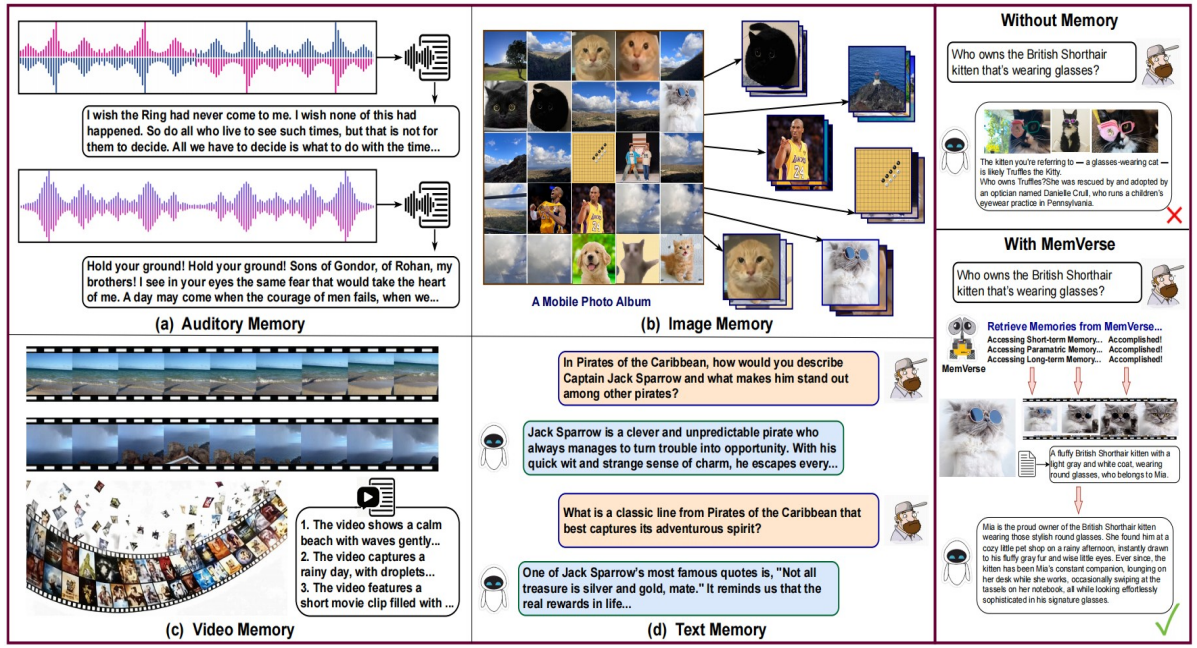
    # Retrieve memory context and add to chat history
    rag_context: str = await user.context(max_token_size=500)
    chat_ctx.add_message(content=rag_context, role="system")

    # Call the default LLM node
    return Agent.default.llm_node(self, chat_ctx, tools, model_settings)
```



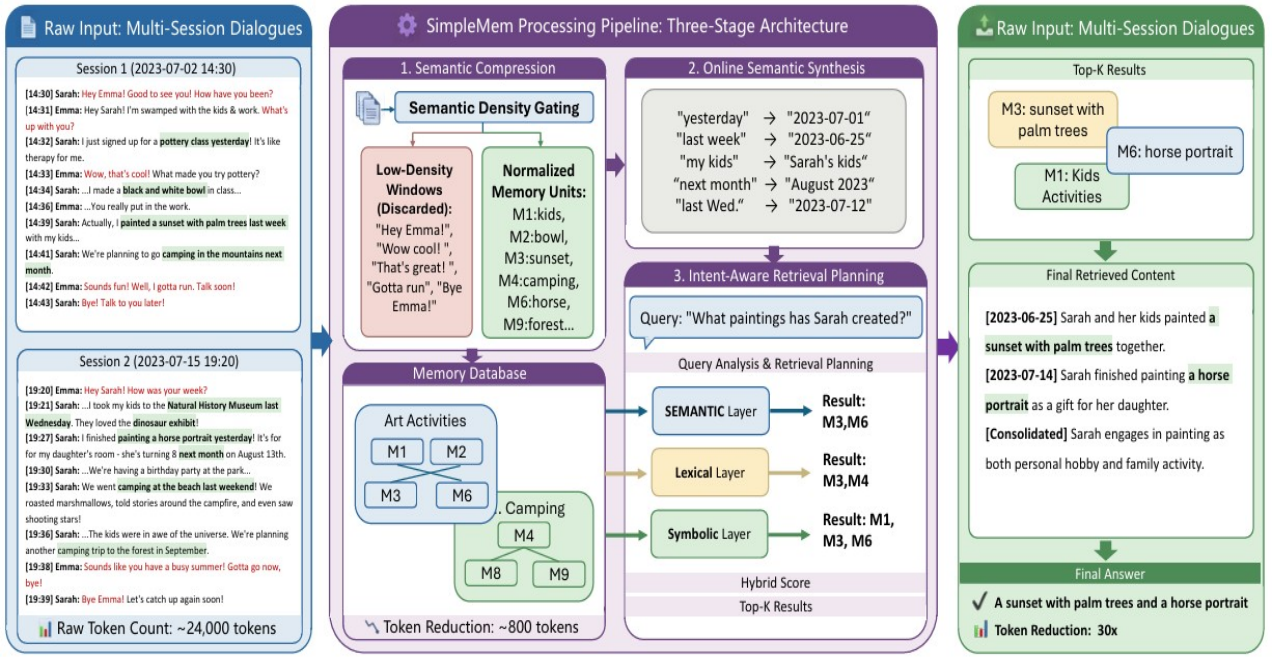
memobase

Memory 框架 - 多模态 Memory



MemVerse

多模态数据向量化，作为多模态原始数据的索引
分层检索，周期性蒸馏



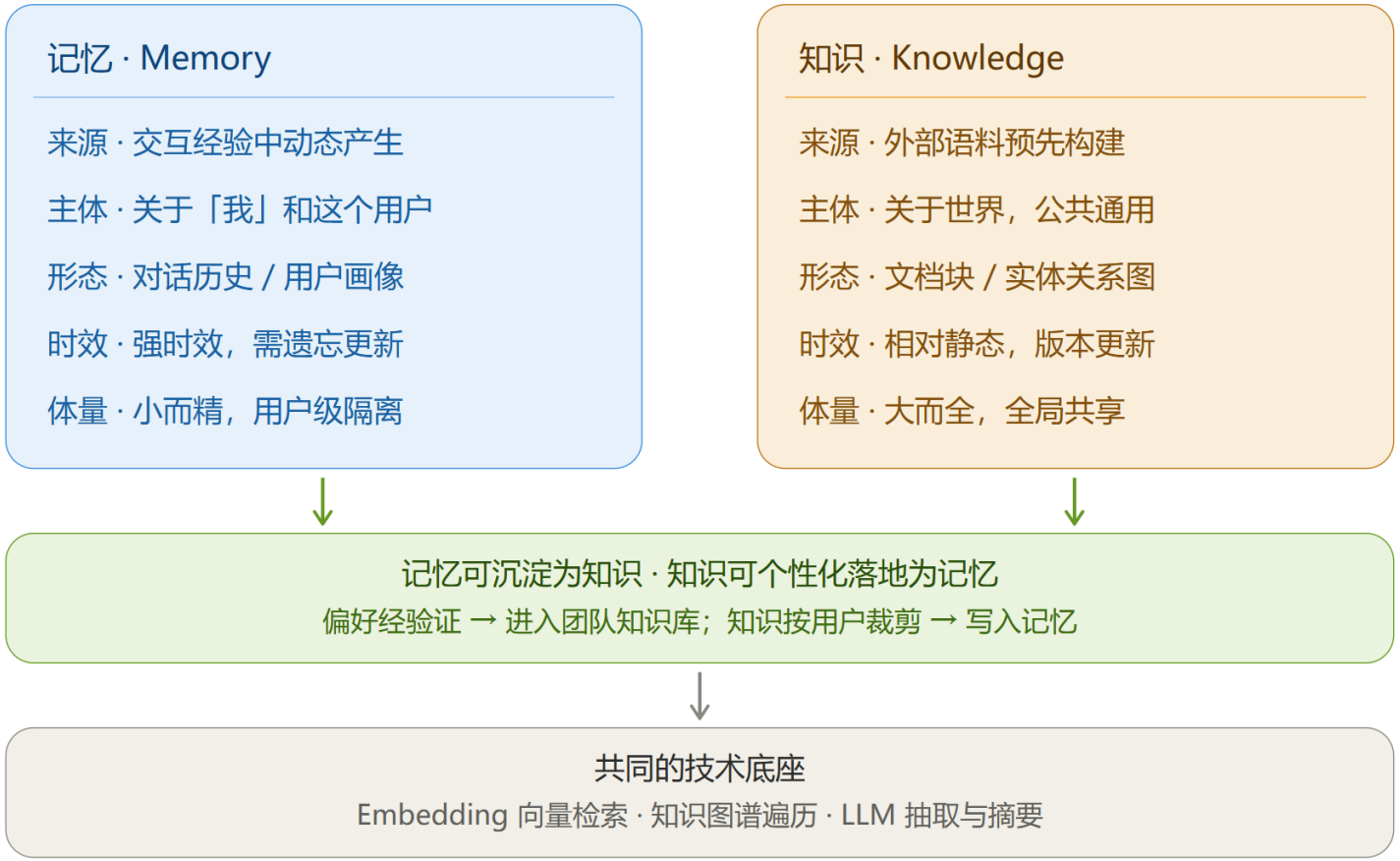
SimpleMem

三阶段流水线：语义结构化压缩、在线语义合成、意图感知检索规划
add_text/add_image/add_audio

Memory Benchmark

基准名称	核心定位	关键特点	主要指标
LongMemEval	长时记忆检索	跨会话、评估 5 项核心长时记忆能力（信息提取、时间推理等）	准确率、召回率
LoCoMo	跨会话推理	侧重从多轮对话中提取事实并执行跨会话时序推理	多跳问答准确率
BEAM	多会话连续性	测试跨多轮会话的连续信息保持与冲突解决能力	一致性得分
AMA-Bench	Agent 轨迹记忆	基于真实 Agent 执行轨迹（含工具调用、代码等）， 评估记忆辅助下的因果推理准确率	推理准确率
STATE-Bench	企业任务表现	在模拟企业流程中测试记忆对任务完成率和执行稳定性的影响	任务完成度
MEMTRACK	多平台状态追踪	在跨平台异步交互（Slack、Linear、Git 等）中， 测试记忆处理冲突、冗余信息的追踪能力	跨平台跨 session 记忆
MUST-Bench	记忆效用决策	测试 Agent 对每条候选记忆的“使用 / 忽略 / 抑制 / 弃权”决策质量	决策准确率
Evo-Memory	记忆自我演化	在流式交互中，评估记忆模块的持续学习与自我演化能力	多轮对话表现

lknowledge



Knowledge Storage

- Neo4j， Cypher 协议，事实标准
- JanusGraph，超大规模， Gremlin 协议
- NebulaGraph，超大规模，毫秒级延迟
- Memgraph，内存图数据库，流式计算
- ArangoDB，提供类 SQL 查询接口
- gStore， RDF 知识图谱

```
CREATE (n:Person {name: 'Alice', age: 30})
CREATE (a:Person {name: 'Alice'})-[r:KNOWS]->(b:Person {name: 'Bob'})

//Alice是否认识Bob
RETURN EXISTS {
  MATCH (a:Person {name: 'Alice'})-[:KNOWS]->(b:Person {name: 'Bob'})
} AS aliceKnowsBob;
```

TongAgent Memory：面向持续交互的分层记忆架构

设计目标：让智能体的重要信息**跨越单次上下文**，在持续交互中**可保留、可更新、可复用**。



以可编辑的记忆块承载人设、用户信息和关键约定。

管理近期交互与工具结果，通过窗口和摘要策略组织本轮推理所需的信息。

关联保留交互记录与原始媒体，支持后续读取与上下文重建。

设计原则 **分层保留** · **按需组织** · **可编程更新**

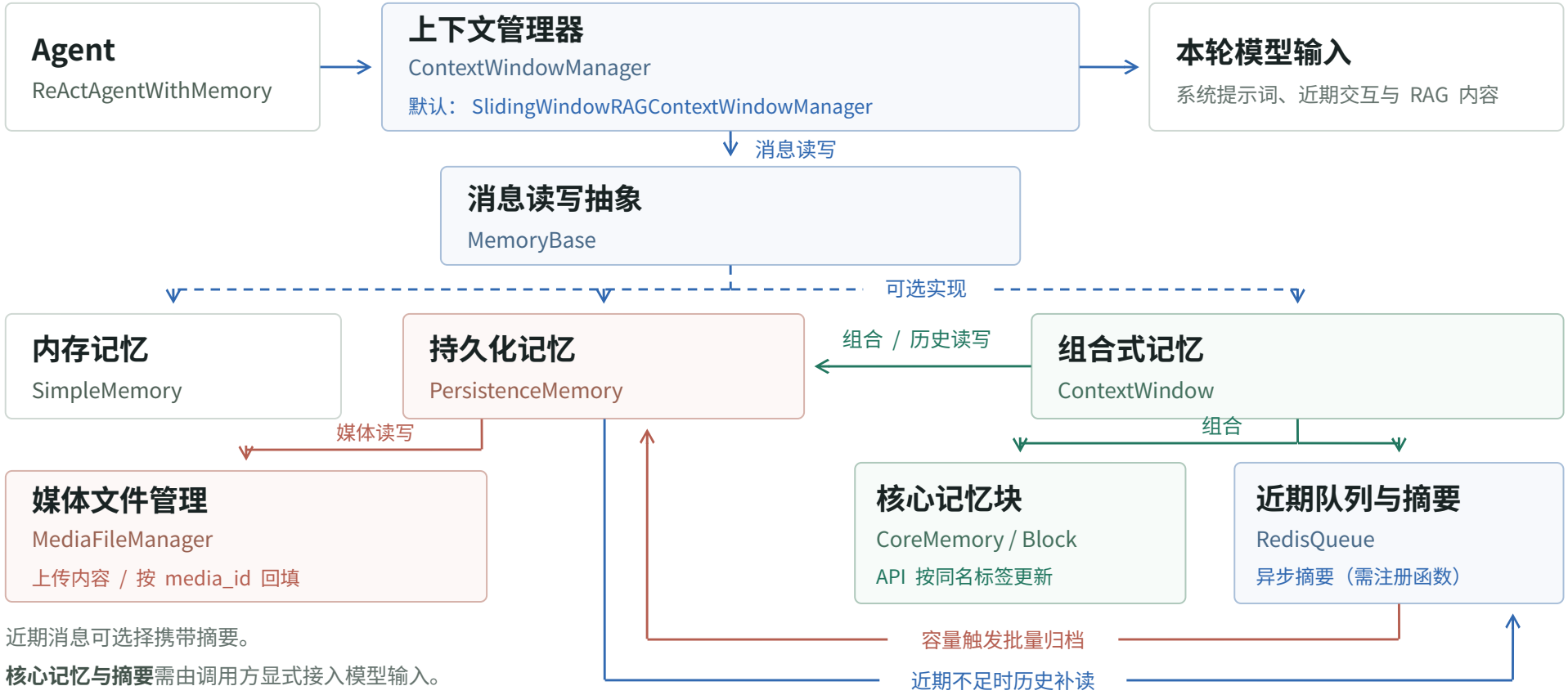
TongAgent Memory：组件化的记忆管理与上下文组织

上下文组装决定**本轮输入**，记忆管理维护**可复用的信息**，通过消息接口衔接。

上下文组装层
生成本轮模型输入

消息记忆接口
统一消息读写

记忆管理层
实现与组件组合



近期消息可选择携带摘要。

核心记忆与摘要需由调用方显式接入模型输入。

SQL 持久化 记忆块、消息记录与媒体引用

Redis 近期消息队列、历史摘要

媒体文件管理 独立存储：本地 / S3

消息接口与实现分离
支持内存、持久化与组合式记忆实现

窗口与摘要可配置
按轮次控制窗口，摘要函数按需注册

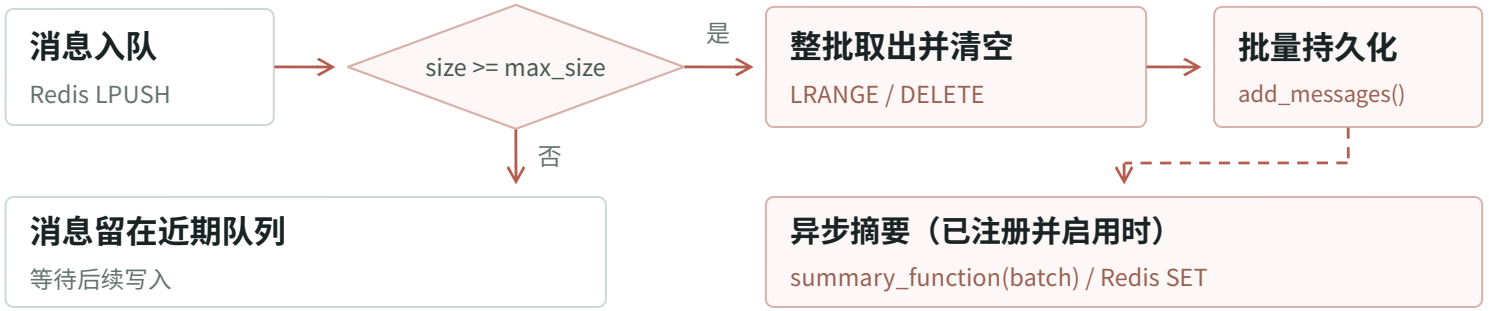
核心记忆可编程更新
通过 API 或工具持续维护记忆块

TongAgent Memory：关键机制与实现细节

围绕**触发条件与数据变化**，展开消息读写、多媒体内容保留和核心记忆更新。

写入：容量触发归档

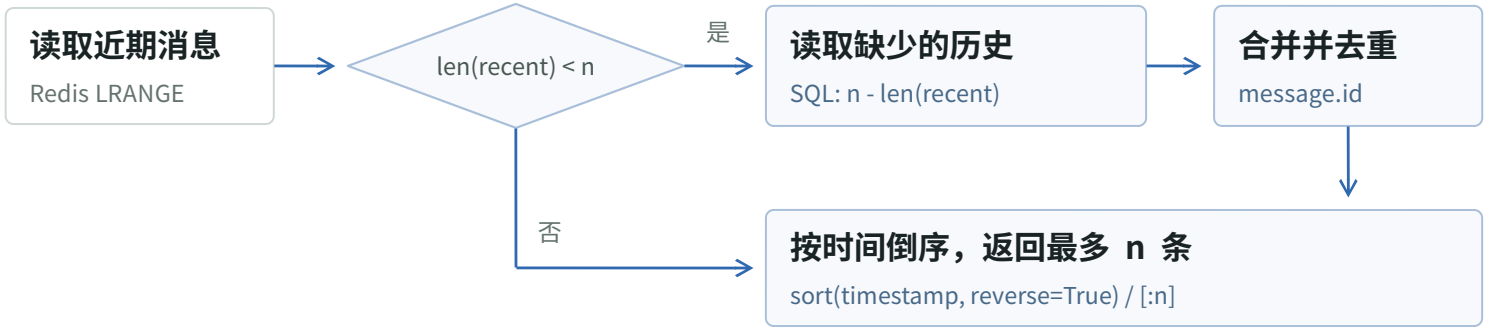
RedisQueue.add_message()



新摘要覆盖旧值，累计融合逻辑由调用方提供。

读取：近期优先，历史补读

RedisQueue.get_messages(n)



可选组装：摘要 + 近期消息 assemble_messages(with_summary=True)

上下文按 ChatHistoryUnit 组织，完成一轮后写入记忆；窗口按轮次裁剪。

多媒体内容的存储与回填

PersistenceMemory.add_message()



require_upload=True 时上传媒体

核心记忆按标签更新

CoreMemory.add_block()



课后习题

作业目标

参考 MemoryBase 与 SimpleMemory，自行实现持久化记忆 FileMemory，接入 ReActAgentWithMemory，使综述 Agent 重启后仍记得上一轮综述。

核心任务

- 实现 FileMemory(MemoryBase)：补全全部抽象方法，记忆以 JSONL 写入本地文件，记录含时间戳、 session 、内容及类型标签
- get_messages 支持按 session 过滤、 recent_n 截断，按类型标签还原为原对象
- 经 LLMRunContext(memory=FileMemory) 注入 ReActAgentWithMemory， search_arxiv 沿用上一题
- 优化 System Prompt：已有综述直接引用、不重复检索；回答注明依据来源

关键要求

读取须还原为原对象（否则框架抛 ValueError）；第 2 轮回答必须来自记忆文件，不得凭模型记忆复述；验收须重启进程，提交记忆文件、第 2 轮上下文与工具调用记录。

最终效果

- 第 1 轮：主题 → search_arxiv → 中文综述 → 写入 FileMemory，退出
- 第 2 轮（重启、同 session）：追问 → 恢复上轮综述 → 不再检索 → 基于综述回答并标注来源